

STEGANOGRAFIE

cu OpenCV

Ascunderea datelor in imagini digitale

C++ / OpenCV

LSB Encoding

MSB Encoding

Bash Payload

Limbaj	C++17
Biblioteca	OpenCV 4.x
Subiect	Image Steganography (LSB & MSB)
Payload	Bash Script ascuns in bitii imaginii
Imagini	PNG (lossless - obligatoriu)

Cuprins

1.	Introducere in Steganografie	3
2.	Arhitectura Proiectului	3
3.	Codare LSB (Least Significant Bit)	4
4.	Codare MSB (Most Significant Bit)	4
5.	Vizualizare & Histograme	5
6.	Extragere si Executie Payload	5
7.	Payload Bash - Analiza Scriptului	6
8.	Comparatie Vizuala a Imaginilor	6
9.	Analiza Securitatii si Detectie	7
10.	Concluzii	7

1. Introducere in Steganografie

Steganografia este arta si stiinta ascunderii informatiilor in interiorul altor date, astfel incat existenta mesajului secret sa nu fie observabila de un tert. Spre deosebire de criptografie, care face datele ilizibile, steganografia le face **invizibile**.

In contextul imaginilor digitale, fiecare pixel este reprezentat prin valori numerice (intensitati de culoare). Tehnicile de steganografie exploateaza faptul ca modificarile minime ale acestor valori sunt imperceptibile ochiului uman, dar pot stoca date binare.

Acest proiect demonstreaza doua tehnici: **LSB (Least Significant Bit)** - care produce imagini vizual identice cu originalul, si **MSB (Most Significant Bit)** - care distorsioneza vizibil imaginea, servind ca referinta educationala.

Caracteristica	Steganografie	Criptografie
Scopul	Ascundere	Protectie
Detectabilitate	Dificila	Evidenta (date cifrate)
Vizibilitate mesaj	Invizibil	Vizibil, dar ilizibil
Securitate	Prin obscuritate	Prin algoritm matematic

2. Arhitectura Proiectului

Proiectul este structurat in jurul unui singur fisier **main.cpp** care expune cinci functii principale:

encodeLSB()	Codifica scriptul bash in bitii LSB ai canalului albastru (B) al imaginii. Modificarile sunt imperceptibile vizual.
encodeMSB()	Codifica scriptul in bitii MSB. Produce distorsiuni vizuale majore - demonstreaza impactul modificarii bitilor importanti.
showRichHistogram()	Calculeaza si afiseaza histogramele RGB ale imaginii originale vs. imaginea stego, pentru comparatie.
showLSBPlane()	Extrage si vizualizeaza planul bitilor LSB ca imagine alb-negru, revelanda pattern-ul datelor ascunse.
extractAndRun()	Extrage scriptul bash din imaginea stego si il executa prin apel system(). Nucleul demonstratiei de atac.

3. Codare LSB - Least Significant Bit

Tehnica LSB functioneaza prin inlocuirea celui mai putin semnificativ bit al valorii unui canal de culoare cu un bit din datele secrete. O modificare de ± 1 din 255 (0.4%) este complet invizibila ochiului uman.

Algoritmul encodeLSB pas cu pas:

- 1. Citirea imaginii PNG cu `imread()` in format BGR (3 canale).
- 2. Citirea scriptului bash ca vector de octeti (bytes) in mod binar.
- 3. Adaugarea unui octet null `\0` la sfarsitul scriptului (terminator).
- 4. Pentru fiecare pixel, se extrage bitul curent din scriptul bash.

- 5. Bitul LSB al canalului albastru (index [0]) este resetat la 0: `pixel &= ~1`
- 6. Noul bit este scris: `pixel |= bit`
- 7. Imaginea modificata este salvata in format PNG (fara pierderi).

Fragment de cod - bucla de encoding:

```
for(int i=0; i<img.rows; i++) {
for(int j=0; j<img.cols; j++) {
int bit = (scriptData[charIndex] >> bitIndex) & 1; // bit curent
img.at<Vec3b>(i,j)[0] &= ~1; // reset LSB la 0
img.at<Vec3b>(i,j)[0] |= bit; // scrie bit din script
if(++bitIndex == 8) { bitIndex=0; charIndex++; }
}
}
```

Capacitate de stocare: O imagine de 512x512 pixeli poate stoca $512 \times 512 / 8 = 32,768$ bytes (~32 KB) de date secrete in canalul LSB.

4. Codare MSB - Most Significant Bit

Tehnica MSB (Most Significant Bit) modifica cel mai important bit al unui octet, cel cu ponderea 128 din 255. Aceasta produce schimbari dramatice si vizibile in imagine, servind ca demonstratie educationala a importantei alegerii bitului corect.

AVERTISMENT: Tehnica MSB are scop EXCLUSIV EDUCATIONAL. Distorsiunile vizuale o fac inutila ca tehnica reala de steganografie - orice observator ar detecta imediat imaginea alterata.

Diferenta fata de LSB:

Caracteristica	LSB	MSB
Bitul modificat	Bitul 0 (valoare 1)	Bitul 7 (valoare 128)
Impact vizual	Imperceptibil (±1)	Extrem de vizibil (±128)
Detectabilitate	Dificila	Imediata
Utilizare practica	Steganografie reala	Numai demonstratie
Cod bit	<code>pixel &= ~1; pixel = bit</code>	<code>pixel &= ~128; pixel = bit<<7</code>

5. Vizualizare & Histograme

Proiectul include doua functii de vizualizare care permit analiza impactului codificarii asupra imaginii: `showRichHistogram()` si `showLSBPlane()`.

`showRichHistogram()` - Analiza distributiei culorilor:

Calculeaza histogramele pentru cele 3 canale (B, G, R) folosind `OpenCV calcHist()` si le normalizeaza. In cazul LSB, histograma ramane aproape identica cu originalul. In cazul MSB, se observa modificari drastice in distributia valorilor.

`showLSBPlane()` - Vizualizarea planului bit:

Extrage bitul LSB al fiecarui pixel si il amplifica la 0 sau 255 pentru vizibilitate. In imaginea originala, planul LSB arata zgomot aleator natural. In imaginea stego LSB, planul revela pattern-uri structurate - semnatura datelor ascunse.

```
// Extragere plan LSB
int lsb = img.at<Vec3b>(i,j)[channel] & 1;
bitPlane.at<uchar>(i,j) = lsb * 255; // 0 sau 255 pentru vizibilitate
```

6. Extragere si Executie Payload

Functia `extractAndRun()` implementeaza procesul invers: citeste imaginea stego, reconstruieste scriptul bash caracter cu caracter, il salveaza pe disc si il executa.

Fluxul de extractie:

1	Citire imagine stego	<code>imread()</code> incarca pixelii imaginii in memorie
2	Reconstructie biti	Bitul LSB al fiecarui pixel este extras si asamblat in octeti
3	Detectie terminator	Bucula se opreste la gasirea octetului null ■
4	Salvare pe disc	Scriptul extras este scris in fisierul <code>extracted.sh</code>
5	Executie	<code>system('chmod +x extracted.sh && ./extracted.sh')</code> ruleaza payload-ul

Nota de Securitate: Utilizarea `system()` pentru executia directa a unui script extras dintr-o imagine reprezinta un vector de atac real. In sisteme de productie, nicio imagine nu ar trebui executata fara validare riguroasa.

7. Payload Bash - Analiza Scriptului

Scriptul `payload.sh` este o demonstratie educationala care simuleaza comportamentul unui malware, fara a produce efecte daunatoare reale.

• Afisare mesaj de compromitere	Simuleaza notificarea unui sistem compromis
• Colectare informatii sistem	Executa <code>date</code> , <code>whoami</code> , <code>pwd</code> - echivalentul recunoasterii
• Creare fisier dovada	Scrie <code>dovada_executie.txt</code> - simuleaza persistenta
• Simulare atac multi-etapa	Mesaje debug despre RSA, reverse shell, C2 (inactive)
• Curatare urme	Simuleaza stergerea logurilor si istoricului bash

Fragment payload (sectiunea de recunoastere):

```
#!/bin/bash
echo 'SISTEM COMPROMIS (DEMO STEGANOGRAFIE)'
echo 'Data si ora executiei: $(date)'
echo 'Utilizator curent: $(whoami)'
echo 'Director de lucru: $(pwd)'
echo '[DEBUG] Stabilire tunel invers - Oprit in modul DEMO.'
echo 'Fisier creat de scriptul ascuns in poza.' > dovada_executie.txt
```

8. Comparatie Vizuala a Imaginilor

Imaginile de mai jos demonstreaza impactul vizual diferit al celor doua tehnici. Imaginea de baza este mandrilul - folosit traditional in procesarea imaginilor (Lena/Mandrill).

[image indisponibila]	[image indisponibila]
Imagine Stego LSB	Imagine Stego MSB
Vizual identica cu originalul. Modificarea LSB este imperceptibila.	Distorsionata vizibil. Culoarele sunt inversate/alterate - detectabila imediat.

Concluzie vizuala: tehnica LSB este complet invizibila, in timp ce MSB produce artefacte cromatice majore datorate modificarii bitului cu ponderea $2^7=128$.

9. Analiza Securitatii si Detectie

Steganografia bazata pe LSB, desi invizibila vizual, poate fi detectata prin tehnici de steganaliza statistica si analiza matematica.

Chi-Square Attack	Analizeaza distributia statistica a valorilor LSB. Datele ascunse produc o distributie mai uniforma decat zgomotul natural.
RS Analysis	Regular-Singular analysis detecteaza modificarile sistematice in grupuri de pixeli adiacenti.
Histogram Analysis	Compararea histogramelor imaginii originale si stego poate revela anomalii subtile in distributia valorilor.
LSB Plane Visual	Vizualizarea directa a planului LSB revela pattern-uri structurate (text/cod) vs. zgomot aleator natural.
File Size Analysis	Imaginile PNG cu date ascunse pot prezenta dimensiuni de fisier usor diferite din cauza compresiei.

Aparare: Formate cu pierderi (JPEG) distrug datele LSB prin recompresie. Sistemele IDS moderne pot detecta imagini stego prin analiza statistica automata. Conversia PNG->JPEG->PNG elimina orice payload ascuns in bitii LSB.

10. Concluzii

Proiectul demonstreaza cu succes principiile fundamentale ale steganografiei in imagini digitale, ilustrand atat tehnici practice cat si limitele lor:

- [V] LSB Encoding este o tehnica eficienta si invizibila pentru ascunderea datelor in imagini PNG lossless.
- [V] Planul bitilor LSB poate fi vizualizat pentru a revela prezenta datelor ascunse - instrument util in analiza forensica.

- [V] Histogramele RGB ofera un prim indicator al modificarilor - utile in detectia steganografiei.
- [!] MSB Encoding demonstreaza importanta alegerii bitului corect - bitii importanti nu pot fi modificati fara a compromite imaginea.
- [X] Executia automata a scripturilor extrase (system()) reprezinta un risc de securitate major in aplicatii reale.
- [i] Formatul PNG este obligatoriu - JPEG ar distruge datele prin recompresie cu pierderi.

Referinte tehnice: OpenCV Documentation (docs.opencv.org) | Provos & Honeyman - 'Hide and Seek: An Introduction to Steganography' (IEEE Security & Privacy, 2003) | Westfeld & Pfitzmann - 'Attacks on Steganographic Systems' (IH 1999)